

Effective Agents Patterns

When to chain, route, parallelize, delegate, critique — and when to ask a human. Every idea traced to the line of Python that implements it.

Xiaowei Ou
Yale Physics Department
Oct 6, 2026

01 chain

02 route

03 parallelize

04 orchestrate

05 evaluate

06 human-in-loop

07 content writer

THE PROGRESSION PATH

Each step adds exactly one idea

Step	Pattern	What it adds	Script
01	Prompt Chaining	Sequential steps	01_prompt_chaining.py
02	Routing	+ input classification	01_routing.py
03	Parallelization	+ concurrent execution	01_parallelization.py
04	Orchestrator-Workers	+ dynamic decomposition	01_orchestrator_workers.py
05	Evaluator-Optimizer	+ self-critique	01_evaluator_optimizer.py
06	Human-in-the-Loop	+ human oversight	01_human_in_the_loop.py
07	Content Writer	Combines all six	content_writer/agent.py

Same blog/content domain throughout, so you can watch the patterns compose. Rule of thumb: **start simple** — reach for orchestration only when chaining isn't enough.

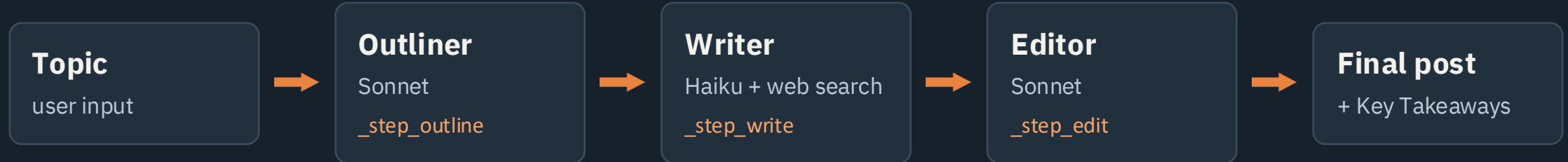
BEFORE THE PATTERNS

Every script shares one skeleton

- 1 Callback, not print**
The agent emits (event, data); main() decides how to render it.
- 2 One flag picks the model**
use_light swaps Sonnet for the cheaper Haiku per call.
- 3 Every call is metered**
One wrapper, so token cost is tracked in one place.
- 4 run() is the pattern**
Read run() in each file; the rest is plumbing.

```
01_prompt_chaining.py
066 ChainCallback = Callable[[str, dict], None] ← 1
069 class PromptChain:
079     def _call_llm(self, system, messages, *,
084         use_light=False, tools=None):
089         model = self.light_model if use_light \ ← 2
            else self.model
096         response = self.client.messages.create(
097             model=model, max_tokens=max_tokens,
099             system=system, messages=messages,
101             **kwargs)
103         self.token_tracker.track(response.usage) ← 3
104         return response
        :
168     def run(self, topic, on_event=None): ← 4
170         self._notify = on_event or (lambda e, d: None)
197 def main(): # UI only: Rich menus
```

A fixed assembly line: each output feeds the next



Chain when...

steps need different models, tools or settings; you want to validate intermediate output; you need to debug each stage.

One prompt when...

one well-crafted prompt handles it reliably. Extra steps only add latency, cost and failure points.

run() is the chain, read top to bottom

1 Output becomes input

topic → outline → draft → final. Each step is a plain variable handed to the next call.

2 Quality gate

An empty outline aborts the chain instead of sending garbage downstream.

3 Traceable steps

step_start / step_complete events give per-stage visibility.

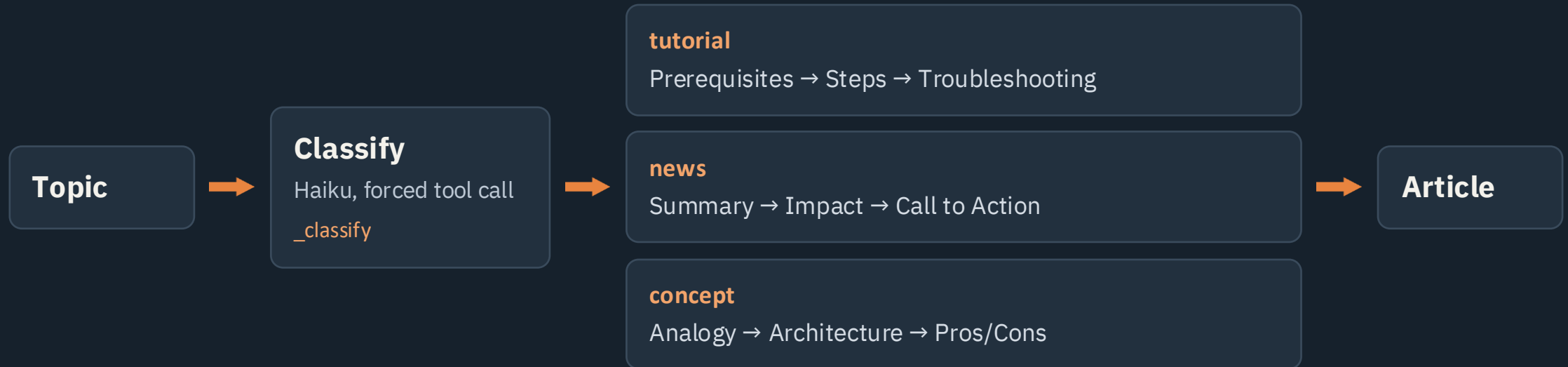
```
01_prompt_chaining.py · PromptChain.run
168 def run(self, topic, on_event=None) -> str:
172 # Step 1: Outline
173 self._notify("step_start", {"name": "Outline"}) ← 3
174 outline = self._step_outline(topic) ← 1
175 if not outline.strip(): ← 2
176     raise ValueError("Outliner produced empty...")
178 self._notify("step_complete", {"name": "Outline", ...})
180 # Step 2: Write
183 draft, searches = self._step_write(outline) ← 1
185 self._notify("step_complete", {"name": "Write", ...})
187 # Step 3: Edit
189 final = self._step_edit(draft) ← 1
193 self._notify("chain_complete", {})
194 return final
```

Each step gets its own model and tools

- 1 Dual-model strategy**
Sonnet where judgment matters, Haiku for straightforward expansion.
- 2 Server-side web search**
A built-in tool; Claude decides when to search, capped by max_uses.
- 3 Only the writer is tooled**
Light model + search on step 2 alone.
- 4 A mini agentic loop**
Keep calling until stop_reason is end_turn (at most 4 more turns).

```
01_prompt_chaining.py
024 MODEL = "claude-sonnet-4-6" ← 1
025 LIGHT_MODEL = "claude-haiku-4-5-20251001"
028 WEB_SEARCH_TOOL = {"type": "web_search_20250305", ← 2
    "name": "web_search", "max_uses": 1}
: # _run_agentic_loop
125 for _ in range(4): ← 4
132     if response.stop_reason == "end_turn": break
136     messages.append({"role": "assistant", ...})
144     messages.append({"role": "user", ...})
146     response = self._call_llm(...)
:
155 def _step_write(self, outline):
157     return self._run_agentic_loop(
159         WRITER_USER_PROMPT.format(outline=outline),
160         use_light=True, ← 3
161         tools=[WEB_SEARCH_TOOL])
```

Classify first, then run the chain built for it



Route when inputs need **structurally different** processing, not just a different tone. Each route *is* a prompt chain from tutorial 01 — so a wrong route means the wrong output format entirely.

A forced tool call is a typed classifier

- 1 An enum is the route list**
Three labels, each defined with an example in the description.
- 2 tool_choice forces it**
The model must call `classify_content`: always valid JSON, no regex.
- 3 Reasoning for transparency**
A required "why" field, shown in the UI.
- 4 Read the dict directly**
`block.input` is already parsed.

```
01_routing.py
037 CLASSIFY_TOOLS = [{"name": "classify_content",
041   "input_schema": {"type": "object", "properties": {
044     "content_type": {"type": "string",
046       "enum": ["tutorial", "news", "concept"], ← 1
047       "description": "tutorial: how-to guides..."},
053     "reasoning": {"type": "string"}}, ← 3
058     "required": ["content_type", "reasoning"]}}}
    :
159 def _classify(self, topic):
162     response = self._call_llm(CLASSIFY_SYSTEM_PROMPT,
165     messages, use_light=True, max_tokens=256,
167     tools=CLASSIFY_TOOLS, ← 2
168     tool_choice={"type": "tool",
169                 "name": "classify_content"})
171     for block in response.content:
172         if block.type == "tool_use": ← 4
173         return block.input
```

The router is a dict; every route is a chain

1 A dispatch table

The classifier's label picks a method. Adding a route = one prompt set + one dict entry.

2 Fail loudly

An unknown label raises instead of guessing.

3 Route = tutorial 01 again

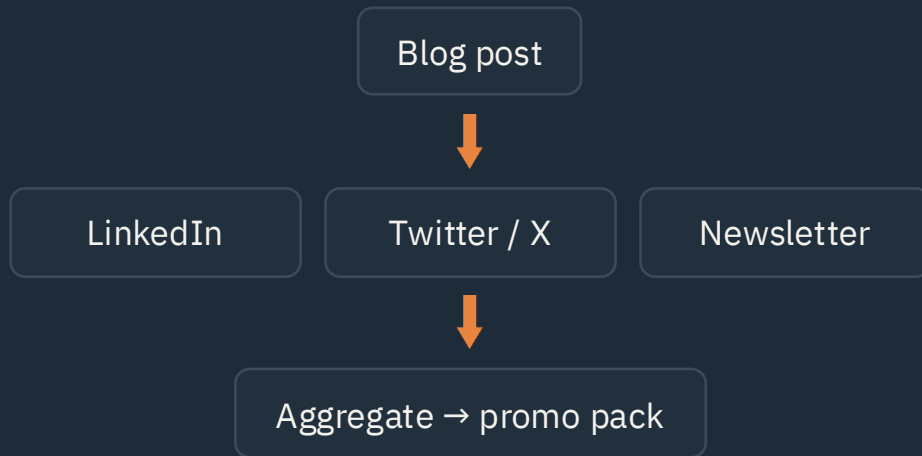
Three chained calls; the middle step runs on Haiku.

```
01_routing.py
177 def _chain_tutorial(self, topic): ← 3
180     prerequisites = self._call_llm_text(PREREQS, ...)
187     steps = self._call_llm_text(STEPS,
189         f"...{prerequisites}...", use_light=True)
195     troubleshooting = self._call_llm_text(..., steps)
200     return f"# {topic}\n\n## Prerequisites..."
        : # run()
277     classification = self._classify(topic)
284     routes = { ← 1
285         "tutorial": self._chain_tutorial,
286         "news": self._chain_news,
287         "concept": self._chain_concept}
290     chain_fn = routes.get(content_type)
291     if not chain_fn: ← 2
292         raise ValueError(f"Unknown content type...")
295     result = chain_fn(topic)
```

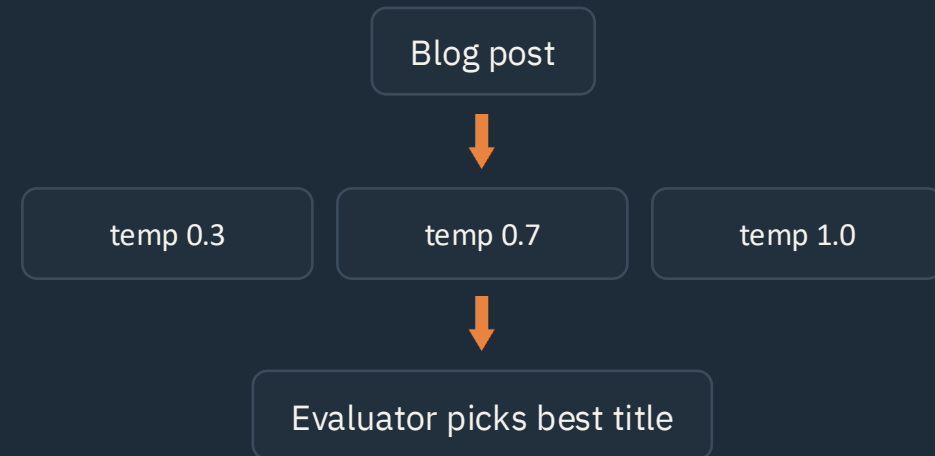
03 · PARALLELIZATION

Fan out independent work, fan in the results

Sectioning · Promo Pack



Voting · SEO title



Rule: tasks must be **truly independent**. If B needs A's output, don't parallelize.

Fan-out is a thread pool over a dict

- 1 Same dict as routing...**
...but instead of picking one entry, we run them all.
- 2 One thread per task**
The Anthropic client is thread-safe; workers = number of tasks.
- 3 Fan-in as they finish**
as_completed yields in finish order, so progress streams live.
- 4 Isolate failures**
One writer failing doesn't crash the fan-out.

```
01_parallelization.py · ParallelContentGenerator.run
129 # Fan-out: run all writers concurrently
131 writers = { ← 1
132     "linkedin": self._write_linkedin,
133     "twitter": self._write_twitter,
134     "newsletter": self._write_newsletter}
137 with ThreadPoolExecutor(max_workers=3) as executor: ← 2
138     futures = {executor.submit(fn, blog_post): name
139                 for name, fn in writers.items()}
139     for future in as_completed(futures): ← 3
140         name = futures[future]
141         try:
142             results[name] = future.result()
143             self._notify("writer_complete", {"name": name})
144         except Exception as e: ← 4
146             results[name] = f"Error: {e}"
```

Voting: vary temperature, let a judge pick

1 Diversity by design

0.3 is safe, 1.0 is surprising. More variety in, better pick out.

2 Same call, N times

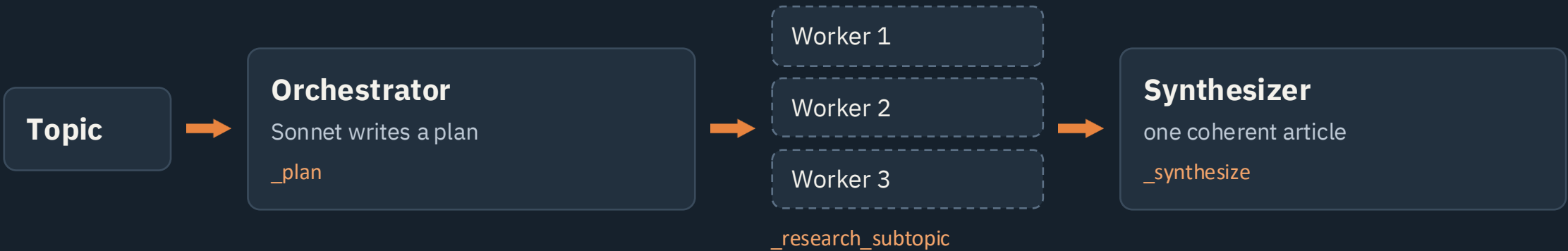
Only the temperature differs; the post is cut to 500 chars to save tokens.

3 An LLM judge

A separate evaluator prompt picks the winner — a preview of tutorial 05.

```
01_parallelization.py
107 def _generate_seo_title(self, blog_post, temperature):
109     return self._call_llm(SEO_TITLE_SYSTEM_PROMPT,
111         f"...{blog_post[:500]}", temperature=temperature)
    : # run()
151 temperatures = [0.3, 0.7, 1.0] ← 1
154 with ThreadPoolExecutor(max_workers=3) as executor:
155     futures_list = [executor.submit( ← 2
156         self._generate_seo_title, blog_post, temp)
        for temp in temperatures]
158 for future in as_completed(futures_list):
160     title = future.result().strip()
161     titles.append(title)
167 # Evaluate and pick the best title
168 if titles: ← 3
170     results["seo_vote"] = \
        self._vote_best_title(titles, blog_post)
```

Now the LLM decides how to split the work



	03 Parallelization	04 Orchestrator-Workers
Who splits the work	You, hardcoded	The LLM, at runtime
Workers	Specialist prompts	Generic researcher + planner's prompt
Merge	Collect results	A dedicated synthesis LLM call

The plan is structured output, too

- 1 **An array = dynamic N**
The model decides how many subtopics (2–4 per the prompt).
- 2 **It writes the workers' prompts**
Each subtopic carries its own research question.
- 3 **...and the merge plan**
synthesis_instructions travel to the synthesizer.
- 4 **Routing's trick, reused**
Forced tool call — but it returns a plan, not a label.

```
01_orchestrator_workers.py
060 PLANNING_TOOLS = [{"name": "create_research_plan",
064   "input_schema": {"properties": {
067     "subtopics": {"type": "array", "items": { ← 1
072     "title": {"type": "string"},
076     "research_prompt": {"type": "string"}}, ← 2
085     "synthesis_instructions": {"type": "string"}}, ← 3
090   "required": ["subtopics", "synthesis_instructions"]}}}
    :
144 def _plan(self, topic):
150   response = self._call_llm(ORCHESTRATOR_SYSTEM_PROMPT,
153   messages, max_tokens=1024, tools=PLANNING_TOOLS,
155   tool_choice={"type": "tool", ← 4
    "name": "create_research_plan"})
158   for block in response.content:
160     return block.input # if tool_use
```

Plan → parallel workers → synthesize

1 The plan sizes the pool

max_workers comes from the LLM's subtopic count, not a constant.

2 Generic worker, given prompt

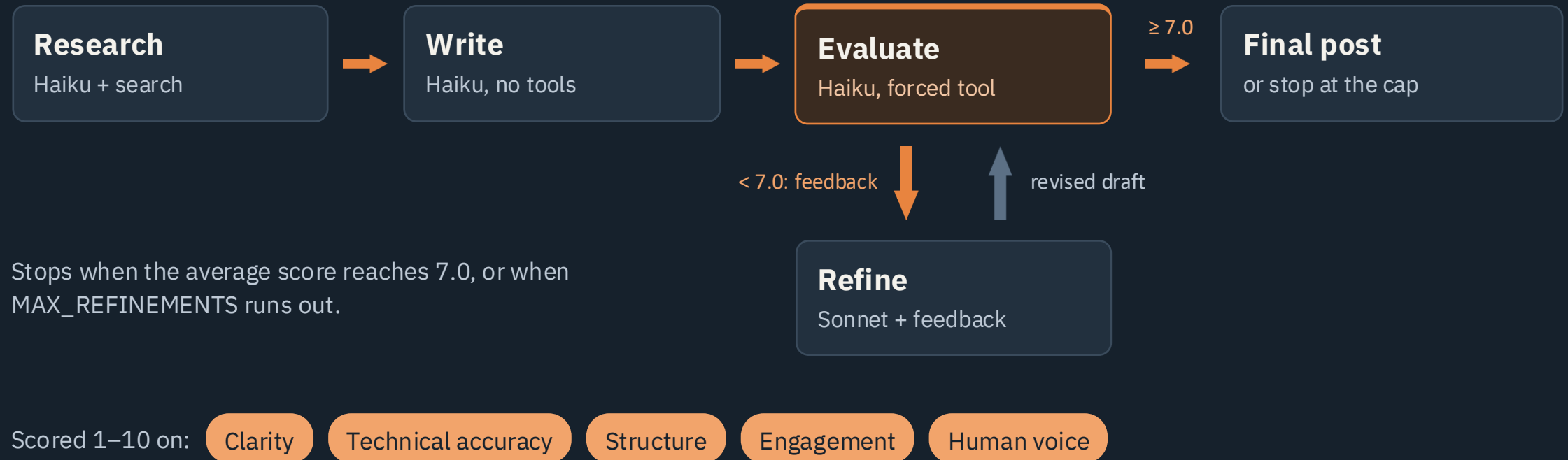
Haiku + web search runs the orchestrator's research_prompt.

3 Synthesis, not concatenation

The planner's instructions are injected into the synthesizer's system prompt.

```
01_orchestrator_workers.py
164 def _research_subtopic(self, subtopic):
169     messages = [{"role": "user", ← 2
                  "content": subtopic["research_prompt"]}]]
170     response = self._call_llm(WORKER_SYSTEM_PROMPT,
171     messages, use_light=True, tools=[WEB_SEARCH_TOOL])
    : # _synthesize
180     system = f'{{SYNTHESIZER_SYSTEM_PROMPT}} ...{{instructions}}' ← 3
    : # run()
194 subtopics = self._plan(topic)["subtopics"] # L193–194
203 with ThreadPoolExecutor( ← 1
    max_workers=len(subtopics)) as executor:
204     futures = {executor.submit(
205         self._research_subtopic, sub): sub["title"] ...}
220 final = self._synthesize(
    topic, research_results, instructions)
```

One model writes, another grades, repeat



A rubric in the prompt, scores in a schema

- 1 **Anchored rubric**
Score bands per dimension, and a demand for specific feedback.
- 2 **Bounded integers**
Five 1–10 fields; compact JSON, not prose.
- 3 **Feedback is data**
issues and suggestions go straight to the refiner.
- 4 **One number to stop on**
The mean of five scores vs the 7.0 threshold.

```

01_evaluator_optimizer.py
064 You are a demanding technical editor... ← 1
066 1. CLARITY: ... (9-10: crystal clear, 7-8: ...)
077 Be specific in feedback: "The intro is generic..."
    :
081 EVALUATION_TOOLS = [{"name": "evaluate_draft",
088   "clarity": {"type": "integer", ← 2
    "minimum": 1, "maximum": 10},
089   # ...accuracy, structure, engagement, voice
098   "issues": {"type": "array", ...}, ← 3
103   "suggestions": {"type": "array", ...}}}]
    :
122 SCORE_THRESHOLD = 7.0
132 def _extract_scores(evaluation):
134     scores = dict(zip(SCORE_DIMENSIONS, ...))
135     return scores, sum(scores.values()) / len(scores) ← 4

```

A bounded loop with a clear exit

1 Converge

Good enough? Stop immediately; don't spend another call.

2 Refine from fresh context

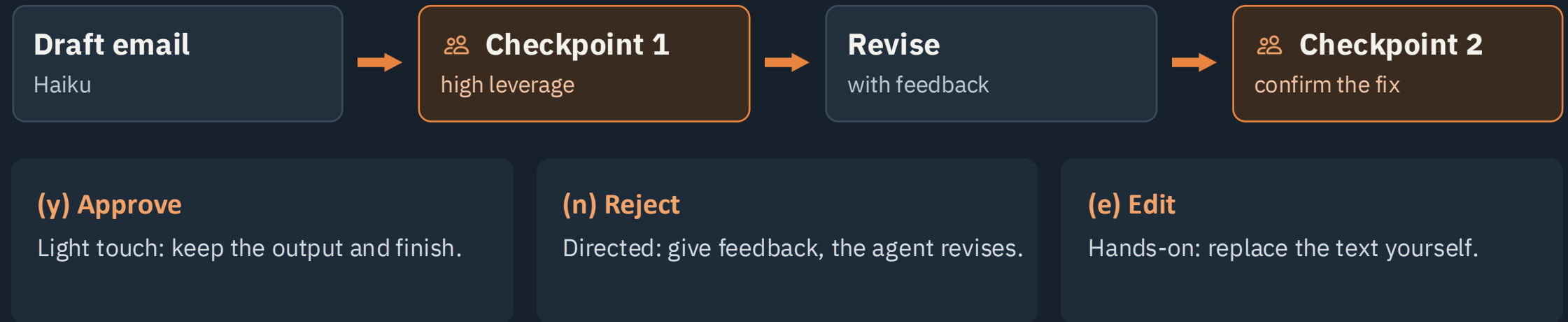
Research + feedback + full draft, no chat history. Sonnet does the rewrite.

3 Hard cap on cost

MAX_REFINEMENTS bounds the loop whatever the scores.

```
01_evaluator_optimizer.py
203 def _refine(self, topic, draft, research, evaluation):
205     feedback = f"Issues: {...['issues']}\n..." ← 2
209     user_msg = f"...{research}...{feedback}...{draft}"
216     return self._call_llm_text(REFINER_SYSTEM_PROMPT, ...)
    : # run()
254 for iteration in range(1, MAX_REFINEMENTS + 1): ← 3
256     evaluation = self._evaluate(draft, topic)
257     scores, avg_score = _extract_scores(evaluation)
271     if avg_score >= SCORE_THRESHOLD: ← 1
272         self._notify("threshold_met", {"avg": avg_score})
273         break
275     if iteration < MAX_REFINEMENTS:
277         draft = self._refine( ← 2
            topic, draft, research, evaluation)
279     else:
280         self._notify("max_iterations", {"avg": avg_score})
```

Pause where a human adds the most leverage



Leverage principle: catching a wrong tone at checkpoint 1 saves all revision work; catching a typo at checkpoint 2 saves nothing.

The human is just a function you pass in

1 Injectable checkpoint

Terminal, test lambda or Slack webhook; defaults to auto-approve.

2 Gate right after the draft

The high-leverage spot, before any revision cost.

3 (bool, str) = three modes

Approve, edit (text replaces draft) or reject.

4 Capped revision loop

Same checks after each revise, at most MAX_REVISIONS.

```
01_human_in_the_loop.py · EmailDrafter
046 CheckpointFn = Callable[[str, str, str], tuple[bool, str]]
    :
078 def run(self, scenario, *, checkpoint_fn=None):
080     check = checkpoint_fn or (lambda *_: (True, "")) ← 1
084     draft = self._draft(scenario)
088     approved, feedback = check("Draft Review", draft, ...) ← 2
094     if approved and not feedback: ← 3
095         return draft # (y) approve
097     if approved and feedback:
098         return feedback # (e) edit: human's text
100     # (n) rejected: enter revision loop
101     for revision in range(1, MAX_REVISIONS + 1): ← 4
103         draft = self._revise(draft, feedback)
107         approved, feedback = check("Revision Review", ...)
113         # same approve / edit returns
119     return draft # max revisions reached
```

Keystrokes in, a (bool, str) tuple out

#	Key	Returns
1	y / Enter	(True, "")
2	e	(True, new text)
3	anything else	(False, feedback)

Swap the UI, keep the agent

main() wires this in on line 186:
 drafter.run(scenario, checkpoint_fn=...). In
 production, replace it with a Slack message or
 UI modal.

01_human_in_the_loop.py · terminal UI

```

122 def human_checkpoint(console, title, content, question):
124     console.print(Panel(content, title=...))
126     console.print("(y)es / (n)o with feedback / (e)dit")
129     response = input().strip().lower()
131     if response in ("y", "yes", ""): ← 1
132         return True, ""
133     elif response.startswith("e"): ← 2
137         # read lines until a blank one
144         return True, "\n".join(lines)
145     else: ← 3
147         feedback = input().strip()
148         return False, feedback

```

Every pipeline phase is a pattern you've seen

Phase	Pattern	From	Model	Method in agent.py
Classify content type	Routing	02	Sonnet	<code>_classify</code>
Plan research subtopics	Orchestrator	04	Sonnet	<code>_plan</code>
Research in parallel	Workers	04	Haiku	<code>_research_section</code>
Write, type-specific voice	Prompt chaining	01	Sonnet	<code>_write</code>
Evaluate + refine loop	Evaluator-Optimizer	05	Sonnet	<code>_evaluate</code>
Human checkpoints	Human-in-the-Loop	06	—	<code>_checkpoint</code>
Social media fan-out	Parallelization	03	Haiku	<code>_write_social</code>
SEO title voting	Voting	03	Haiku + Sonnet	<code>_generate_seo</code>

About 15–22 API calls per topic. Social + SEO run only after the final human approval — no tokens spent on rejected articles.

run_stream(): six patterns, one generator

Badges = tutorial numbers. Follow them down the code.

- 02 Routing** — classify the type
- 06 Human** — confirm or override it
- 04 Orchestrator** — plan, parallel research
- 01 Chaining** — write from research
- 05 Evaluator** — score, refine if < 7.0
- 03 Parallel** — social fan-out, SEO vote

Sync calls run via `asyncio.to_thread`, so the generator can keep yielding events.

```
content_writer/agent.py · run_stream
463 yield ClassifyStartEvent()
464 classification = await to_thread(self._classify, ...) ← 02
469 approved, feedback = await to_thread(_checkpoint, ← 06
471 "classification", ...)
486 subtopics = await to_thread(self._plan, topic, ...) ← 04
494 def _research_parallel(): # thread pool
508 sections = await to_thread(_research_parallel)
523 draft, srcs = await to_thread(self._write, ...) ← 01
531 evaluation = await to_thread(self._evaluate, ...) ← 05
535 if evaluation.avg_score < score_threshold: # refine
575 approved, user_feedback = await to_thread( ← 06
577 _checkpoint, "final_review", ...)
643 if human_approved:
646 social = await to_thread(self._write_social, ...) ← 03
655 seo = await to_thread(self._generate_seo, draft)
```

Typed data and typed events, end to end

- 1 Pydantic, not loose dicts**
 Tool-call JSON is validated: a 12 for clarity fails loudly.
- 2 Derived values on the model**
 avg_score replaces tutorial 05's helper function.
- 3 One class per event**
 A Literal stage tag discriminates the union.
- 4 UI pattern-matches**
 The callback idea from slide 3, now type-safe.

```

content_writer/models.py
057 class EvaluationResult(BaseModel):
060     clarity: int = Field(ge=1, le=10) ← 1
065     issues: list[str] = []
068     @computed_field ← 2
070     def avg_score(self) -> float: .../ 5
121 class ClassifyDoneEvent(BaseModel):
124     stage: Literal["classify_done"] = "classify_done" ← 3
125     classification: ClassificationResult
259 AgentEvent = Union[ClassifyStartEvent, ...]

01_content_writer.py · async_main
193 async for event in agent.run_stream(topic, ...): ← 4
199     match event:
204         case ClassifyDoneEvent(classification=c):
205             console.print(f"✓ {c.content_type.value}...")

```

Production concerns: rate limits and cost

- 1 Retry only on 429s**
 Parallel phases can burst past limits; other errors still surface.
- 2 Exponential backoff**
 30 s to 120 s waits, up to 6 attempts.
- 3 Bound every search**
 One search per worker; each injects ~25–35k tokens.
- 4 Truncate where it's cheap**
 Haiku side-tasks see 2000/500 chars; Sonnet sees it all.

```

content_writer/agent.py
104 @retry(
105     retry=retry_if_exception_type(RateLimitError), ← 1
106     wait=wait_exponential(multiplier=2, min=30, max=120), ← 2
107     stop=stop_after_attempt(6),
108     before_sleep=before_sleep_log(logger, log_level=20))
110 def _call_api(self, **kwargs):
112     return self.client.messages.create(**kwargs)

content_writer/tools.py
019 # max_uses controls searches per phase
023 "max_uses": 1, ← 3

content_writer/agent.py
348 f"Create a LinkedIn post...{article[:2000]}" ← 4
394 f"Generate an SEO title for:...{article[:500]}"

```

CHOOSING A PATTERN

Start simple; climb only when the problem asks

If your problem has...	Reach for	Mechanism in the code
Steps needing different setups	Prompt chaining	output → next input, gates
Structurally different inputs	Routing	forced tool_choice + dict
Independent subtasks	Parallelization	thread pool + as_completed
A split known only at runtime	Orchestrator-workers	plan as structured output
Clear quality criteria	Evaluator-optimizer	rubric, threshold, hard cap
Costly mistakes	Human-in-the-loop	injectable checkpoint fn

TAKEAWAYS

Three habits running through all seven scripts

Force structure

tool_choice turns labels, plans, scores and votes into JSON you can trust.

Bound the cost

Right-size models, cap loops and searches, truncate side inputs.

Decouple the UI

Callbacks, then typed events: the agent never prints.

Further reading

[Building Effective Agents — Anthropic](#)

Agentic Design Patterns — Andrew Ng, The Batch

Questions?